Rapport de validation — tests et donnees

Ce notebook contient deux parties :

1. **Le validateur fonctionne-t-il ?** — Tests unitaires sur des images synthetiques pour verifier que le code detecte bien chaque cas problematique.
2. **Mes donnees sont-elles pretes ?** — Validation de vraies images (`data/test_images/`) pour voir ce que le pipeline recevra.

Partie 1 — Le validateur fonctionne-t-il ?

On lance les memes tests que `make test` et on visualise chaque cas.

Resultat pytest

```
import subprocess
result = subprocess.run(
    ["python", "-m", "pytest", "tests/", "-v", "--tb=short"],
    capture_output=True, text=True,
    cwd="/root/projet_cancer_sein"
)
print(result.stdout)
if result.returncode != 0:
    print(result.stderr)
```

```
============================= test session starts ==============================
platform linux -- Python 3.13.12, pytest-9.0.2, pluggy-1.5.0 -- /root/miniconda3/bin/python
cachedir: .pytest_cache
rootdir: /root/projet_cancer_sein
```

```
configfile: pyproject.toml
plugins: anyio-4.10.0
collecting ... collected 14 items

tests/test_validate_input.py::TestCSV::test_valid_csv PASSED              [  7%]
tests/test_validate_input.py::TestCSV::test_missing_csv PASSED            [ 14%]
tests/test_validate_input.py::TestCSV::test_csv_is_a_directory PASSED     [ 21%]
tests/test_validate_input.py::TestCSV::test_missing_columns PASSED        [ 28%]
tests/test_validate_input.py::TestCSV::test_invalid_laterality PASSED     [ 35%]
tests/test_validate_input.py::TestCSV::test_invalid_view PASSED           [ 42%]
tests/test_validate_input.py::TestImages::test_valid_grayscale PASSED     [ 50%]
tests/test_validate_input.py::TestImages::test_rgb_image_rejected PASSED [ 57%]
tests/test_validate_input.py::TestImages::test_tiny_image_warns PASSED    [ 64%]
tests/test_validate_input.py::TestImages::test_black_image_rejected PASSED [ 71%]
tests/test_validate_input.py::TestImages::test_corrupted_file PASSED      [ 78%]
tests/test_validate_input.py::TestImages::test_white_image_rejected PASSED [ 85%]
tests/test_validate_input.py::TestImages::test_large_image_warns PASSED   [ 92%]
tests/test_validate_input.py::TestImages::test_uint8_valid PASSED         [100%]

============================== 14 passed in 3.94s ==============================
```

Tests CSV

Le validateur verifie que le fichier `train.csv` est lisible, complet et coherent.

Test	Ce qui est simule	Resultat attendu
`test_valid_csv`	CSV correct avec 8 lignes, colonnes valides	`passed=True`, 8 lignes lues
`test_missing_csv`	Chemin vers un fichier inexistant	`passed=False`, erreur « introuvable »
`test_csv_is_a_directory`	Un dossier nomme `train.csv` (cas reel rencontre sur le poste)	`passed=False`, erreur « dossier »
`test_missing_columns`	CSV avec seulement `patient_id` et `image_id`	`passed=False`, erreur « Colonnes manquantes »
`test_invalid_laterality`	Valeur `Left` au lieu de `L`	`passed=False`, erreur « Lateralites invalides »
`test_invalid_view`	Valeur `XRAY` au lieu de `CC` ou `MLO`	`passed=False`, erreur « Vues invalides »

Tests images

Pour chaque cas, une image est generee (ou lue depuis `data/demo`) et soumise au validateur.

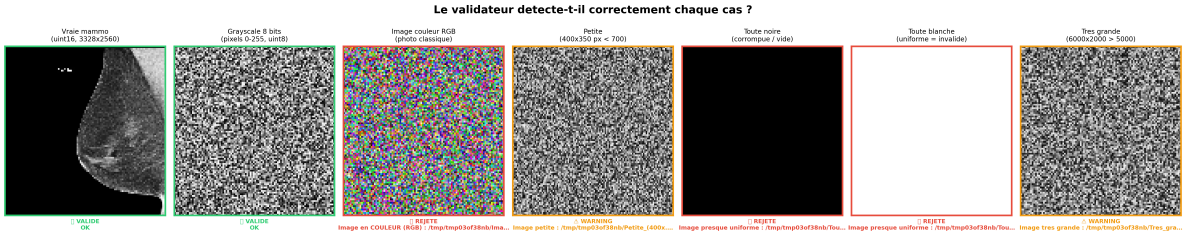
Seuils appliques :

- **Taille recommandee : 700 x 700 px** — en dessous, warning (le pipeline continue).
- **Warning > 5000 px sur une dimension** — l'image est acceptee mais un avertissement est emis.
- **Image uniforme** (toute noire ou toute blanche) : rejetee si l'ecart min-max < 50.

```
import os, sys, tempfile
import cv2
import numpy as np
import matplotlib.pyplot as plt

sys.path.insert(0, "/root/projet_cancer_sein/scripts")
from validate_input import ValidationResult, check_image

tmp = tempfile.mkdtemp()

REAL_MAMMO = "/root/projet_cancer_sein/data/demo/train_images/13101/447716546.png"

cases = []

# 1. Vraie mammographie (uint16, depuis data/demo)
img = cv2.imread(REAL_MAMMO, cv2.IMREAD_UNCHANGED)
cases.append(("Vraie mammo\n(uint16, 3328x2560)", img, "valide"))

# 2. Uint8 valide (distribution 0-255, entiers)
img = np.random.randint(0, 256, (3000, 2000), dtype=np.uint8)
cases.append(("Grayscale 8 bits\n(pixels 0-255, uint8)", img, "valide"))

# 3. RGB
img = np.random.randint(0, 255, (3000, 2000, 3), dtype=np.uint8)
cases.append(("Image couleur RGB\n(photo classique)", img, "erreur"))

# 4. Petite (warning seulement, pas erreur)
img = np.random.randint(10, 240, (400, 350), dtype=np.uint8)
cases.append(("Petite\n(400x350 px < 700)", img, "warning"))
```

```python
# 5. Toute noire
img = np.zeros((3000, 2000), dtype=np.uint8)
cases.append(("Toute noire\n(corrompue / vide)", img, "erreur"))

# 6. Toute blanche
img = np.full((3000, 2000), 255, dtype=np.uint8)
cases.append(("Toute blanche\n(uniforme = invalide)", img, "erreur"))

# 7. Tres grande (warning, pas erreur)
img = np.random.randint(10, 240, (6000, 2000), dtype=np.uint8)
cases.append(("Tres grande\n(6000x2000 > 5000)", img, "warning"))

fig, axes = plt.subplots(1, len(cases), figsize=(20, 4))
fig.suptitle("Le validateur detecte-t-il correctement chaque cas ?", fontsize=13, weight="

for ax, (title, img_array, expected) in zip(axes, cases):
    fname = title[:12].replace("\n", "_").replace(" ", "_").replace("x", "x") + ".png"
    path = os.path.join(tmp, fname)
    cv2.imwrite(path, img_array)
    result = ValidationResult()
    ok = check_image(path, result)

    display = img_array if img_array.ndim == 2 else img_array[:, :, ::-1]
    cmap = "gray" if display.ndim == 2 else None
    thumb = display[::max(1, display.shape[0]//100), ::max(1, display.shape[1]//100)]
    vmax = 65535 if display.dtype == np.uint16 else 255
    ax.imshow(thumb, cmap=cmap, aspect="auto", vmin=0, vmax=vmax)
    ax.set_title(title, fontsize=8)

    if result.errors:
        status = "  REJETE"
        color = "#e74c3c"
        msg = result.errors[0][:45] + "…"
    elif result.warnings:
        status = "  WARNING"
        color = "#f39c12"
        msg = result.warnings[0][:45] + "…"
    else:
        status = "  VALIDE"
        color = "#2ecc71"
        msg = "OK"
```

```
    ax.set_xlabel(f"{status}\n{msg}", fontsize=7, color=color, weight="bold")
    ax.set_xticks([])
    ax.set_yticks([])
    for spine in ax.spines.values():
        spine.set_edgecolor(color)
        spine.set_linewidth(2)

plt.tight_layout()
plt.show()
```



Test	Ce qui est simule	Resultat attendu
`test_valid_grayscale`	Vraie mammographie uint16 depuis `data/demo`	`True`, aucune erreur
`test_uint8_valid`	PNG 3000x2000 uint8, pixels 0-255 (entiers)	`True`, aucune erreur
`test_rgb_image_rejected`	PNG 3000x2000 en couleur RGB	`False`, erreur « COULEUR »
`test_tiny_image_warns`	PNG 400x350 px (sous le seuil 700px)	`True` mais warning « petite »
`test_black_image_rejected`	PNG entierement noir (image corrompue)	`False`, erreur « uniforme »
`test_white_image_rejected`	PNG entierement blanc	`False`, erreur « uniforme »
`test_large_image_warns`	PNG 6000x2000 (dimension > 5000px)	`True` mais warning « grande »
`test_corrupted_file`	Fichier `.png` contenant du texte brut	`False`, erreur « illisible »

Partie 2 — Mes donnees sont-elles pretes ?

Ces images de `data/test_images/` simulent ce qu'un utilisateur pourrait soumettre par erreur (photos, screenshots, fichiers web). C'est l'equivalent visuel de `make validate`.

```python
import os, sys
import cv2
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

sys.path.insert(0, "/root/projet_cancer_sein/scripts")
from validate_input import ValidationResult, check_image

images_dir = "/root/projet_cancer_sein/data/test_images"
files = sorted(os.listdir(images_dir))

if not files:
    print("Aucune image dans data/test_images/")
else:
    SUPPORTED = (".png",)
    UNSUPPORTED = (".jpg", ".jpeg", ".webp", ".avif", ".bmp", ".tiff", ".gif", ".heic", ".

    fig, axes = plt.subplots(1, len(files), figsize=(14, 5))
    if len(files) == 1:
        axes = [axes]
    fig.suptitle("Vos images - le validateur les accepte-t-il ?", fontsize=12, weight="bol

    for ax, fname in zip(axes, files):
        path = os.path.join(images_dir, fname)
        ext = os.path.splitext(fname)[1].lower()
        short_name = fname[:30] + "…" if len(fname) > 30 else fname

        try:
            pil_img = Image.open(path).convert("RGB")
            pil_img.thumbnail((300, 300))
            ax.imshow(np.array(pil_img))
        except Exception:
            ax.text(0.5, 0.5, "Apercu\nindisponible", ha="center", va="center",
                    transform=ax.transAxes, fontsize=9)
            ax.set_facecolor("#f0f0f0")

        if ext in UNSUPPORTED:
            status = "  FORMAT NON SUPPORTE"
            msg = f"Extension {ext} rejetee\n(seuls PNG/DICOM sont acceptes)"
            color = "#e74c3c"
        elif ext in SUPPORTED:
```

```
        result = ValidationResult()
        ok = check_image(path, result)
        if ok:
            status = "  VALIDE" if not result.warnings else "  WARNING"
            msg = result.warnings[0][:50] if result.warnings else "Image acceptee"
            color = "#f39c12" if result.warnings else "#2ecc71"
        else:
            status = "  REJETE"
            msg = result.errors[0][:50] + "…" if result.errors else ""
            color = "#e74c3c"
    else:
        status = "  INCONNU"
        msg = f"Extension {ext} non reconnue"
        color = "#f39c12"

    ax.set_title(short_name, fontsize=7.5, wrap=True)
    ax.set_xlabel(f"{status}\n{msg}", fontsize=7.5, color=color, weight="bold")
    ax.set_xticks([])
    ax.set_yticks([])
    for spine in ax.spines.values():
        spine.set_edgecolor(color)
        spine.set_linewidth(2.5)

plt.tight_layout()
plt.show()

print("\nDIAGNOSTIC - data/test_images/\n" + "=" * 50)
for fname in files:
    ext = os.path.splitext(fname)[1].lower()
    if ext in UNSUPPORTED:
        verdict = f"[FAIL]  Format non supporte ({ext})"
    elif ext in SUPPORTED:
        result = ValidationResult()
        ok = check_image(os.path.join(images_dir, fname), result)
        if ok and result.warnings:
            verdict = f"[WARN]  {result.warnings[0][:70]}"
        else:
            verdict = "[OK]   Valide" if ok else f"[FAIL]  {result.errors[0][:70]}"
    else:
        verdict = f"[WARN]  Extension inconnue ({ext})"
    print(f"  {fname[:45]:<45}  {verdict}")
```

```
DIAGNOSTIC - data/test_images/
==================================================
  Blanc.png                                       [FAIL]  Image RGBA (avec transparence) : /ro
  color_splash.png                                [FAIL]  Image en COULEUR (RGB) : /root/proje
  images.jpg                                      [FAIL]  Format non supporte (.jpg)
  photo-stone-texture-pattern_58702-16052.avif    [FAIL]  Format non supporte (.avif)
  shooting-anniversaire-bebe-un-an-garcon-decor   [FAIL]  Format non supporte (.webp)
  wilderness_picture_cute_small_squirrel_branch   [FAIL]  Format non supporte (.jpg)
```